

บทที่ 2

Object-Oriented Concept

2.1 บทนำ

วิธีการเขียนโปรแกรมแบบโครงสร้าง (Structure Programming) ซึ่งมีการใช้งานกันอย่างแพร่หลาย เป็นการเขียนที่มีข้อดีเหนือกว่าการพัฒนาระบบโดยไม่ใช้โมเดลใด ๆ แต่อย่างไรก็ดีก็ยังมีปัญหาหลักก็คือการที่โมเดลในการออกแบบพัฒนาระบบไม่เชื่อมต่อไปถึงรายละเอียดในการพัฒนาระบบ แผนภาพต่าง ๆ ที่ใช้อธิบายการไหลของข้อมูล(Data flow) ก็ไม่ได้บอกถึงรูปแบบในการเขียนโปรแกรม บอกแต่เพียงว่าระบบมีการทำงานอย่างไร มีข้อมูลใดใช้ในกระบวนการไหน มีการเคลื่อนย้ายของข้อมูลอย่างไรบ้าง ซึ่งผู้ออกแบบและพัฒนาระบบใช้แผนภาพ Data Flow Diagram (DFD) ต่อมาก็ได้มีผู้คิดค้นและพัฒนาโมเดลการพัฒนาระบบที่เรียกว่าการพัฒนาระบบเชิงวัตถุ (Object-Oriented Development)

การพัฒนาระบบเชิงวัตถุประกอบไปด้วย 2 ขั้นตอน คือ ขั้นตอนของการวิเคราะห์และออกแบบระบบเชิงวัตถุ (Object-Oriented Analysis and Design) ซึ่งมีโมเดลที่เป็นมาตรฐานมารองรับคือ Unified Modeling Language (UML) ซึ่งได้อธิบายไปแล้วในบทต้น ๆ ขั้นตอนต่อมาจะเข้ามาสู่การพัฒนาระบบซึ่งในขั้นตอนนี้ใช้หลักการสำหรับการเขียนโปรแกรมเชิงวัตถุ (Object-Orient Programming Concept) ซึ่งจะกล่าวถึงในหัวข้อต่อไป

2.2 วัตถุ (Object)

ออบเจกต์เป็นชนิดของข้อมูลซึ่งประกอบด้วยกลุ่มของข้อมูลที่เรียกว่าแอตทริบิวต์(Attribute)และกลุ่มของฟังก์ชันการทำงาน(Operation) โดยการใช้ข้อมูลและฟังก์ชันที่มีอยู่เหล่านี้จึงจะมาสารททำงานหนึ่ง ๆ ได้เสร็จสมบูรณ์ การเขียนโปรแกรมเชิงวัตถุ คือการสร้างและการเรียกใช้ออบเจกต์ให้ทำงานตามที่เรต้องการ ในการเรียกใช้ออบเจกต์เราจะสนใจเฉพาะการทำงานของออบเจกต์โดยไม่ต้องสนใจรายละเอียดของออบเจกต์

คุณสมบัติของวัตถุ

การรวมข้อมูลเข้ากับฟังก์ชัน (Encapsulation) คือการรวมโครงสร้างข้อมูล (Data structure) กับฟังก์ชันที่เกี่ยวกับการเรียกใช้หรือเข้าถึงข้อมูลนั้น (Method) ทำให้เกิดออบเจกต์ที่สามารถซ่อนข้อมูลของตัวเองได้ การกำหนดโครงสร้างของข้อมูลและฟังก์ชันการทำงานจะถูกกำหนดโดยคลาส (Class)

สามารถการสืบทอด (Inheritance) คือการกระจายหรือสืบทอดคุณสมบัติของโครงสร้างข้อมูลและฟังก์ชันที่เป็นคุณสมบัติของออบเจกต์หนึ่งไปยังอีกออบเจกต์หนึ่ง การสืบทอดมีหลายรูปแบบซึ่งจะกล่าวถึงต่อไป

สามารถมีหลายรูปแบบ (Polymorphism) คือการที่ออบเจ็กต์ต่าง ๆ มีการทำงานที่ไม่ขึ้นต่อกัน ที่นี้เพราะการที่แต่ละออบเจ็กต์มีสถานะเป็นของตนเอง ทำให้ผลของการทำงานของแต่ละออบเจ็กต์ไม่เหมือนกันถึงแม้จะเรียกใช้งานแบบเดียวกัน

2.3 ลักษณะที่สำคัญของแนวคิดเชิงวัตถุ

2.3.1 การกำหนดการเข้าถึงข้อมูลภายในคลาส

ในการรักษาค่าหรือข้อมูลภายในคลาส จะมีการกำหนดการเข้าถึงข้อมูลภายในคลาสไว้ 3 ระดับ คือ

2.3.1.1 การกำหนดระดับการเข้าถึงแบบ Private การประกาศหรือกำหนดตัวแปรหรือฟังก์ชันที่เป็นแบบ Private นี้ เป็นการบอกว่าการเรียกใช้หรือเข้าถึงตัวแปรหรือฟังก์ชันด้วยกระบวนการใด ๆ จะเรียกใช้ได้ภายในคลาสของตัวเองเท่านั้น ไม่สามารถเรียกใช้ข้ามคลาสได้ ดูตัวอย่างโค้ดภาษาซี

<pre>Class book { int B_id; void setB_id (int Id); };</pre>	<pre>Class book { private: int B_id; void setB_id (int Id); };</pre>
---	--

จากตัวอย่างข้างต้น ตัวแปร B_id และฟังก์ชัน setB_id (Int Id) ได้ประกาศเป็นแบบ private เมื่อเราสร้างออบเจ็กต์ของคลาสนี้ เราจะไม่สามารถเรียกใช้ตัวแปร B_id และฟังก์ชัน setB_id ได้ เพราะออบเจ็กต์เป็นกระบวนการที่อยู่ภายนอกคลาสดังนี้

<pre>Book B1; B1.B_id = 10001; B1.setB_id (1001);</pre>	<pre>// Can't assign. // Can't assign.</pre>
---	--

2.3.1.2 การกำหนดระดับการเข้าถึงแบบ Public การเข้าถึงตัวแปรหรือฟังก์ชันจะตรงกันข้ามกับแบบ Private คือการเรียกใช้จะใช้ได้ทั้งกระบวนการที่อยู่ในคลาสเดียวกันหรืออยู่ต่างคลาสนั้น ซึ่งรวมถึงออบเจ็กต์ของคลาสดังนี้ จึงถือได้ว่าเป็นการประกาศสมาชิกแบบสาธารณะ ดูตัวอย่างโค้ด

```

Class pub_book {
    Int B_id;
    Public:
    void setB_id (int Id) { B_id = Id;}
};

```

จากตัวอย่างข้างบนจะเห็นว่า ยังคงกำหนดการเข้าถึงตัวแปร B_id เป็นแบบ private ทำให้เราไม่สามารถที่จะเข้าถึงตัวแปร ได้โดยกระบวนการภายนอกคลาส แต่ฟังก์ชัน setB_id () ซึ่งสามารถเรียกใช้ตัวแปร B_id ได้นั้น เพราะว่าฟังก์ชัน setB_id เป็นการบวนการภายในคลาส

ดังนั้นเราสามารถที่จะกำหนดค่าให้กับตัวแปร B_id ภายในคลาส pub_book ได้ ถึงแม้ว่าจะมีการประกาศตัวแปร B_Id เป็นแบบ private ก็ตาม โดยการเรียกผ่านฟังก์ชัน setB_id () อีกทีหนึ่ง

```

pub_book b1;
b1.setB_id (10001);           // Can assign

```

เมื่อฟังก์ชัน setB_id () ของออบเจ็กต์ b1 รับค่าพารามิเตอร์ 10001 เข้าไป กระบวนการภายในคลาสนี้เริ่มขึ้น ฟังก์ชัน setB_id () ซึ่งเป็นฟังก์ชันแบบ public จะกำหนดค่าพารามิเตอร์ที่อยู่ในตัวแปร Id ให้กับตัวแปร B_Id ที่เป็นแบบ private ได้

จะเห็นได้ว่าเมื่อเรากำหนดการเข้าถึงตัวแปรหรือฟังก์ชันภายในคลาสเป็นแบบ public จะสามารถเรียกใช้ตัวแปรหรือฟังก์ชันได้ แม้ว่าจะเป็นกระบวนการภายนอกคลาสนี้ก็ตาม

2.3.1.3 การกำหนดระดับการเข้าถึงแบบ protected การประกาศสมาชิกคลาสแบบ protected จะมีลักษณะการเข้าถึงเช่นเดียวกับแบบ private แต่ต่างกันที่จะเพิ่มสิทธิให้แก่สมาชิกของคลาสลูกในการเข้าถึงตัวแปรแบบ protected ในคลาสแม่ได้ ในขณะที่ระดับ private ไม่ยอมให้คลาสใด ๆ เข้าถึงตัวแปรได้เลย ดูตัวอย่างโค้ด

```

Class pro_book {
    protected:
        int B_id;
};

class In_book:public pro_book {
    public:
    void setB_id (int Id) { B_id = Id;}
}

```

ตัวอย่างข้างต้นเป็นการประกาศตัวแปร B_id ของคลาส pro_book เป็นแบบ protected ซึ่งเมื่อทำการสืบทอดไปเป็นคลาส In_book ซึ่งเป็นคลาสลูก ตัวแปร B_id ก็สามารถใช้ได้โดยกระบวนการภายในคลาสลูกนี้ได้โดยใช้ฟังก์ชัน setB_id ()

```
in_book b1;
b1.setB_id (2000);           //Can assign.
```

เมื่อเราสร้างออบเจกต์ ของคลาส In_book ออบเจกต์จะไม่สามารถเข้าถึงตัวแปร B_id ได้โดยตรง แต่สามารถเรียกใช้ฟังก์ชัน setB_id () เพื่อกำหนดค่าให้กับตัวแปรต่าง ๆ ให้กับออบเจกต์ของคลาส In_book ได้

การกำหนดการเข้าถึงในการประกาศตัวแปรหรือฟังก์ชันที่แตกต่างกันใน 3 ระดับข้างต้น จะมีผลต่อการสืบทอดของคลาสซึ่งจะกล่าวถึงในข้อถัดไปด้วย

2.3.2 การสืบทอดของคลาส (Inheritance)

เป็นการสร้างคลาสใหม่โดยใช้โครงสร้างข้อมูลและฟังก์ชันจากคลาสที่มีอยู่แล้ว โดยการสร้างคลาสใหม่โดยวิธีการนี้เรียกว่าการสืบทอด คลาสที่เป็นต้นแบบ เรียกว่าคลาสแม่ คลาสที่เป็นผลลัพธ์จากการสืบทอดเรียกว่าคลาสลูก การสืบทอดคลาสลูกจะยังคงคุณสมบัติของคลาสแม่ทุกประการ แต่อย่างไรก็ตามการสืบทอดจะถูกกำหนดระดับการเข้าถึงสมาชิกดังที่ได้กล่าวแล้วข้างต้น และสามารถเพิ่มคุณสมบัติใหม่ ๆ เข้ามาในคลาสอีกก็ได้ เช่น เพิ่มตัวแปร , เพิ่มฟังก์ชัน ลักษณะการประกาศการสืบทอดคลาส ดูจากตัวอย่างได้

```
#include <string.h>

Class book {
public:
    int B_id;
    void setB_id (int Id);
};

class In_book:public book{
    string B_name[50];
    void createB_name (string name[50]);
};
```

จากตัวอย่างข้างบน คลาส In_book ได้จากการสืบทอดจากคลาส book ซึ่งสืบทอดในลักษณะ public กล่าวคือ คลาส In_book สามารถเรียกใช้ตัวแปรหรือฟังก์ชันที่เป็น public หรือ protected จากคลาส book ได้ และมีการเพิ่มตัวแปร B_name ซึ่งเป็นตัวแปรชนิดสตริงและฟังก์ชัน createB_name () ในคลาส In_book อีกด้วย